v1.0.0
20260302

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[Lab Notes]
THREAD SAFETY

THREAD SAFETY DEFINITION

- A central aspect of concurrent programming is writing **thread-safe** code / thread-safe classes La thread-safety è la capacità di un pezzo di codice di funzionare esattamente come previsto, anche quando " bombardato" da più thread contemporaneamente, senza che l'utente debba aggiungere protezioni esterne.
  - a class is *thread-safe* if *it continues to behave correctly when accessed from multiple threads*
    - a prescindere da ~~**regardless of**~~ **the scheduling or interleaving of the execution of those threads by the runtime environment**
    - **with no additional synchronization or other coordination on the part of the calling code**
- *Correctness* means that a class è conforme alle proprie specifiche ~~*conforms to its specification*~~
  - a good specification defines
    - **invariants** constraining an object's state sono regole sullo stato dell'oggetto che devono essere sempre vere.
    - **post-conditions** describing the effects of its operations È il risultato atteso dopo un'operazione.
- Thread-safe classes encapsulate any needed synchronization so that clients need not to provide their own

SHARED MUTABLE STATE

Per avere un bug di concorrenza, devono verificarsi tre condizioni contemporaneamente:
- Stato: Hai dei dati (variabili, oggetti).
- Condiviso (Shared): Più thread possono "vedere" e toccare quei dati.
- Mutabile (Mutable): Quei dati possono cambiare nel tempo.

Se elimini anche solo uno di questi elementi, il problema sparisce. Ecco perché le tre soluzioni proposte sono le uniche vie d'uscita:

- Writing thread-safe code is – at its core – about *managing access to state and in particular to* **shared, mutable state**:
  - **shared**: variable or object could be accessed by multiple threads
  - **mutable**: its value could change during its lifetime
- if multiple threads access the same mutable state variable without appropriate synchronization, the program is *broken*
  - race conditions
- There are three ways to fix it:
  1. don't share state variable across threads
  2. make the state variable immutable
  3. use synchronization whenever accessing the state variable

Se devi per forza condividere dati che cambiano, devi usare la sincronizzazione.
Come funziona: Si usano meccanismi come Lock, Mutex o Semafori. Quando un thread vuole accedere al dato, acquisisce un "permesso" esclusivo. Gli altri thread devono mettersi in coda finché il primo non ha finito.
Il costo: È la soluzione più complessa perché, se gestita male, può rallentare il programma (overhead) o causare blocchi totali (Deadlock).

Un oggetto immutabile è un oggetto che, una volta creato, non cambia mai più il suo valore (pensa a una String in Java o a una costante).
Perché funziona: Se dieci persone leggono lo stesso libro, ma nessuno può scriverci sopra o strappare pagine, non ci sarà mai conflitto.
In pratica: Si usano classi con campi final o readonly e non si forniscono metodi "setter".

Se un dato è accessibile da un solo thread alla volta, non serve sincronizzazione.
Esempio: Una variabile locale all'interno di un metodo vive nello stack di quel thread specifico. Nessun altro thread può vederla, quindi è sicura per definizione.
In pratica: Si usano pattern come il ThreadLocal in Java o architetture "Single Threaded" per la gestione dei dati.

STATELESS & STATEFUL OBJECT

- *Stateless* objects are *always* thread-safe
  - actions of a thread accessing a stateless object cannot affect the correctness of operations in other threads
- Thread safety is ~~undermined as soon as~~ compromessa non appena we share and access in ~~R/W~~ read/write stateful objects
  - **mutable** ~~state-full~~ objects con stato (state-full), in particular

Un oggetto è stateful quando ha variabili interne che possono cambiare nel tempo (stato mutabile). Qui iniziano i problemi di thread-safety.

RACE CONDITIONS

il risultato finale del programma dipende da chi arriva primo e da come si intrecciano le loro operazioni (La Race Condition avviene quando questa sequenza "non-atomica" viene interrotta da un altro thread che modifica gli stessi dati nel mezzo della sequenza.).

- The concurrent execution of non-atomic sequence of statements that should be considered atomic generate **race conditions**
  - occur when the *correctness of a computation depends on the relative timing or interleaving of multiple threads* by the runtime (...and getting the right answer relies on lucky timing..)
- Main examples
  - **lost updates**
  - **check-and-act**

LOST UPDATES & CHECK-AND-ACT

- **Lost updates**
  - when executing concurrently non-atomic read-modify-write operations
    - ex: count++
- **Check-and-act**
  - when a potentially state observation is used to make a decision on what to do next
    - since check+act are not atomic, the state can change after check and before act.
  - example:

```
if (file X doesn't exist) -- check
    then create file X      -- act
```

- *Compound atomic actions*
  - *check-and-act* and *read-modify-write* are examples of compound actions
    - sequences of operations that must be executed atomically in order to remain thread-safe

ATOMIC COMPOUND ACTIONS IN JAVA : SYNCHRONIZED BLOCKS

- Compound-actions - and atomic statement blocks - in Java can be realised by means of synchronized blocks or methods

```
synchronized(lock){
    statement
    statement
    statement
}
```

- A synchronized block has 2 parts
  - a reference to an object that will serve as the lock
  - block of code to be guarded by the lock
- Mostly used at a *method* level
  - synchronized attribute
  - more about this in next modules when discussing monitors in Java

INTRINSIC LOCK AND ENTRY SET

- Atomic blocks work by exploiting the **lock** embedded in each Java object (more on this in next modules)
  - called **intrinsic lock** or monitor lock
  - functioning as a *guard* for the block
- The lock is automatically acquired and then released by a thread respectively when entering and exiting the block
  - if the lock is already acquired, the thread is blocked (suspended) and added to the entry set
  - when a thread exited the block, one thread of the entry set is selected and re-activated
  - no ordering policy is specified
  - if the lock is not released by the thread inside the block, threads in the entry set are blocked forever (starvation)
- For static methods and fields, the lock is associated to the related Class object
- For **synchronized** methods, the object serving as lock is **this**

LOCK REENTRANCY (1/2)

Il concetto di Lock Reentrancy è fondamentale per evitare una situazione paradossale: un thread che blocca se stesso causandosi un deadlock.

- def: **lock reentrancy**
  - when locks are acquired on a *per-thread* basis
    - vs. per-invocation basis
    - per-invocation basis is adopted instead as default locking behavior for Pthreads (POSIX threads) mutex-es
- Java intrinsic locks are reentrant:
  - if a thread tries to acquire a lock that it already holds, the request succeeds

- Per-invocation (Non-reentrant): Il lock viene acquisito per ogni singola richiesta. Se un thread possiede già il lock e prova a richiederlo di nuovo, il sistema vede una nuova richiesta di acquisizione. Poiché il lock è già occupato, il thread viene messo in attesa. Ma dato che il thread sta aspettando se stesso, rimarrà bloccato per sempre. Default in Pthreads (C/C++): Se non diversamente specificato, i mutex POSIX si comportano così.
- Per-thread (Reentrant): Il lock viene acquisito dal thread. Se il thread che già possiede il lock tenta di acquisirlo nuovamente, il sistema dice: "Sei sempre tu, prego, passa pure". Default in Java: Gli intrinsic locks (quelli usati con la keyword synchronized) sono rientranti.

LOCK REENTRANCY (2/2)

- Reentrancy facilitates encapsulation of locking behaviour and ~~thus~~ quindi simplify the development of OO concurrent code

```
public class Widget {
  public synchronized doSomething(){...}
}

public class LoggingWidget extends Widget {
   public synchronized void doSomething(){
     System.out.println(toString()+": calling doSomething");
     super.doSomething();
   }
}
```

- Without reentrancy the above example would lead to a deadlock

PERFORMANCE: POOR CONCURRENCY PROBLEM

Quando un thread entra in un blocco synchronized (o acquisisce un lock), tutti gli altri thread che hanno bisogno dello stesso lock devono fermarsi. Se crei blocchi atomici troppo grandi (chiamati Coarse-grained locks), costringi i thread a procedere uno alla volta. In pratica, hai pagato il costo di gestire più thread (memoria, gestione del contesto), ma ottieni la velocità di un singolo thread.

- The misuse of atomic blocks can lead to performance problems
  - sequentialisations => strong impact on speed-up
    - poor performances
- From a performance perspective, it is important to do a careful choice of what parts must be designed and implemented as critical sections
  - practical rule: minimise the time in which a lock is retained
  - …but don’t trade performance with safety

Se riduci troppo il blocco atomico, potresti inavvertitamente lasciare fuori un'operazione che doveva essere parte del blocco atomico, riaprendo la porta alle Race Conditions.

Per massimizzare le performance, devi fare in modo che il "lucchetto" rimanga chiuso per il minor tempo possibile. Ecco come si fa in pratica:
- Riduci l'ampiezza (Scope): Non sincronizzare l'intero metodo se solo una riga accede allo stato condiviso.
- Sposta le operazioni lente FUORI dal lock: Operazioni come I/O (leggere un file), chiamate di rete o calcoli matematici pesanti su variabili locali non dovrebbero mai stare dentro un blocco synchronized.

BIBLIOGRAPHY

- *“Java Concurrency in Practice”*, Brian Goetz, Addison Wesley